Mini project report on

GIT REPOSITORY FORENSICS

A Database-Driven Git Repository Intelligence Platform

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

KRISHITHA KANKANALA	**PES2UG24CS234**
SARAH KAZI	**PES2UG24CS224**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

This is to certify that the mini project entitled

Git Repository Forensics: A Database-Driven Git Repository Intelligence Platform

is a bonafide work carried out by

KRISHITHA KANKANALA	**PES2UG24CS234**
SARAH KAZI	**PES2UG24CS224**

In partial fulfilment for the completion of fifth semester DBMS Project (UE23CS351A) in the Program of Study – Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Git Repository Forensics** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

KRISHITHA KANKANALA	**PES2UG24CS234**
SARAH KAZI	**PES2UG24CS224**

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A – DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

TABLE OF CONTENTS

No.	Section
–	Certificate, Declaration, Acknowledgement
–	List of Tables and List of Figures
1	Title
2	Students' SRN and Name
3	Abstract
4	Tech Stack
5	SQL – Database Commands
6	NoSQL / Vector Database Commands
7	Next.js – Project Tree Structure
8	Sample Screenshots of Application Code and Demo
9	Evaluation Rubrics – Completion Status
–	References / Bibliography
A	Appendix A – Definitions, Acronyms and Abbreviations

LIST OF TABLES

Section	Title
2	Students' SRN, name and contribution
4	Technology stack
4	Architecture components
6	Redis commands used in the application
7	Role of the main folders
9	Evaluation rubrics – completion status

LIST OF FIGURES

Figure	Title
1	Relational schema – commits table with generated column, CHECK constraints and GIN full-text index
2	bus_factor() stored function using window functions
3	Full-text search and pgvector similarity search queries
4	Neo4j Cypher query for reviewer recommendation
5	Redis read-through cache with graceful fallback
6	Commit-message embedding by feature hashing
7	Sign-in page
8	Repositories – register a repository and see analysis status
9	Overview – summary cards, bus factor, monthly activity (materialised view) and top hotspots
10	Developers – per-developer activity and ownership
11	Hotspots – revisions, authors and churn
12	Change coupling filtered by minimum Jaccard similarity
13	Team analytics – core contributors and shared files (GROUP BY … HAVING)
14	Full-text commit search (tsvector, ranked and highlighted)
15	Semantic commit search (pgvector cosine similarity)
16	Knowledge graph (Neo4j) – collaboration pairs and knowledge silos
17	File detail – history, owners, coupled files and suggested reviewers
18	Analysis jobs with live progress
19	Administration – users, roles and audit log

1. TITLE

Git Repository Forensics

A Database-Driven Git Repository Intelligence Platform

Project type	DBMS Experiential Learning – Level 3
Course	UE23CS351A – DBMS Project, B.Tech CSE, PES University
Domain	Database Management Systems
Databases used	PostgreSQL 17 (relational + pgvector), Neo4j 5 (graph), Redis (cache)
Source code	https://github.com/krista9669/git-repository-forensics
Academic term	AUG – DEC 2026

2. STUDENTS' SRN AND NAME

Sl. No.	Name	SRN	Contribution
1	KRISHITHA KANKANALA	PES2UG24CS234	Database design and core analysis: relational schema and constraints, Git history parsing, change-coupling and embedding algorithms, Neo4j projection, database / Redis / Neo4j integrations, authentication
2	SARAH KAZI	PES2UG24CS224	Application development and schema evolution: Next.js UI, API route handlers, background worker, migration 002 and team-analytics queries, migration / seed scripts, tests and documentation

3. ABSTRACT

Software repositories hold a rich history of commits, authorship and file changes, but Git's own commands make it hard to answer higher-level questions: who really knows a given file, which files are risky to change, which files secretly depend on each other, and how dependent the project is on a few people.

Git Repository Forensics is a database-driven platform that mines a Git repository's history and stores it in a normalised PostgreSQL schema of 13 tables. From that data it computes developer contribution and ownership (using git blame), code hotspots (revisions, authors and churn), change coupling between files (co-change count, directional confidence and Jaccard similarity) and the repository's bus factor. Team analytics such as core contributors and shared files are produced with SQL aggregation, GROUP BY … HAVING, subqueries and window functions.

Three database technologies each handle the workload they suit best. PostgreSQL is the source of truth and also provides full-text search (tsvector with a GIN index) and similarity search over commit messages using the pgvector extension with an HNSW index. Neo4j holds a rebuildable graph of developers and files and answers reviewer-recommendation, collaboration and knowledge-silo questions with Cypher. Redis provides a read-through cache, live job-progress tracking and rate limiting.

Analysis runs asynchronously: a worker claims queued jobs using FOR UPDATE SKIP LOCKED, mines the repository, replaces old results in one transaction, refreshes a materialised view and rebuilds the Neo4j projection. Data integrity is enforced in the database through primary and foreign keys, CHECK constraints, generated columns, partial unique indexes, three triggers, a stored function and a stored procedure. The interface is a Next.js (TypeScript) application with session-based authentication, role-based access, an audit log and a dashboard of charts and reports. The project is covered by 58 unit and integration tests.

Keywords: Git mining, PostgreSQL, pgvector, Neo4j, Redis, change coupling, bus factor, Next.js.

4. TECH STACK

Layer	Technology	Purpose
Frontend + Backend	Next.js 15 (App Router), React 19	Server-rendered pages and JSON API route handlers in one application
Language	TypeScript 5.9, Node.js 22+	Application, worker and scripts
Relational database	PostgreSQL 17	Source of truth: schema, constraints, views, functions, triggers, full-text search
Vector database	pgvector 0.8 (HNSW index)	Cosine-similarity search over 256-dimension commit-message embeddings
Graph database (NoSQL)	Neo4j 5.26 Community	Developer–file graph, reviewer recommendation, collaboration and knowledge silos
Key-value store (NoSQL)	Redis 7.4	Report cache with TTL, job progress, rate limiting
Drivers / libraries	pg, neo4j-driver, ioredis, zod, bcryptjs	Database access, input validation, password hashing
UI	Tailwind CSS 3, Recharts	Styling and charts
Data source	Git 2.30+ (log, blame)	History and ownership extraction
Infrastructure	Docker Compose	Runs PostgreSQL, Neo4j and Redis locally
Testing	Vitest 3	Unit and integration tests

Why this combination?

- **PostgreSQL** gives relational integrity, transactions, joins, window functions and advanced indexing.
- **pgvector** adds vector search inside the same database, avoiding a separate vector store.
- **Neo4j** expresses multi-hop developer-to-file-to-file traversals far more directly than SQL joins.
- **Redis** serves frequently requested reports and live job state quickly; the application still works if Redis is down.
- **Next.js** lets the UI and API live in one TypeScript codebase.

Architecture

Component	Responsibility
Next.js app + JSON API	Pages, authentication, report endpoints
Background worker	Claims jobs, mines Git, computes analytics, writes to PostgreSQL, projects to Neo4j
PostgreSQL + pgvector	Authoritative storage and text / vector search
Neo4j	Rebuildable graph projection of PostgreSQL data
Redis	Cache (10-minute TTL), job progress (24 h TTL), rate-limit counters

5. SQL – DATABASE COMMANDS

All SQL is PostgreSQL 17. The schema lives in two versioned migrations (db/migrations/001_schema.sql and 002_team_analytics.sql) applied inside transactions and tracked in the schema_migrations table. Application queries use bound parameters ($1, $2, …).

5.1 Setup commands

```
npm run db:up          # docker compose: PostgreSQL 17 + pgvector, Neo4j, Redis
npm run db:migrate     # apply 001_schema.sql, 002_team_analytics.sql
npm run neo4j:init     # create Neo4j constraints and indexes
npm run db:seed -- --demo
```

5.2 DDL – types, tables and indexes

Extension and enumerated types

```
CREATE EXTENSION IF NOT EXISTS vector;


CREATE TYPE user_role   AS ENUM ('admin', 'analyst');
CREATE TYPE repo_source AS ENUM ('remote', 'local');
CREATE TYPE job_status  AS ENUM ('queued', 'running', 'succeeded', 'failed', 'cancelled');
CREATE TYPE change_kind AS ENUM ('A', 'M', 'D', 'R', 'C', 'T');
```

Users and sessions

```
CREATE TABLE users (
    id             BIGSERIAL PRIMARY KEY,
    email          TEXT        NOT NULL,
    display_name   TEXT        NOT NULL CHECK (length(display_name) BETWEEN 1 AND 100),
    password_hash  TEXT        NOT NULL,
    role           user_role   NOT NULL DEFAULT 'analyst',
    is_active      BOOLEAN     NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT users_email_format CHECK (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')
);
CREATE UNIQUE INDEX users_email_lower_uq ON users (lower(email));


CREATE TABLE sessions (
    token_hash  TEXT        PRIMARY KEY,
    user_id     BIGINT      NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at  TIMESTAMPTZ NOT NULL,
    CHECK (expires_at > created_at)
);
CREATE INDEX sessions_user_idx    ON sessions (user_id);
CREATE INDEX sessions_expires_idx ON sessions (expires_at);
```

Repositories and analysis jobs

```
CREATE TABLE repositories (
    id                BIGSERIAL   PRIMARY KEY,
    name              TEXT        NOT NULL CHECK (length(name) BETWEEN 1 AND 200),
    source_type       repo_source NOT NULL,
    source            TEXT        NOT NULL,
    branch            TEXT,
    created_by        BIGINT      REFERENCES users(id) ON DELETE SET NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_analyzed_at  TIMESTAMPTZ,
    head_sha          CHAR(40),
    CONSTRAINT repositories_source_uq UNIQUE (source_type, source),
    CONSTRAINT repositories_remote_https CHECK (source_type <> 'remote' OR source LIKE
'https://%')
);
```

```
CREATE TABLE analysis_jobs (
    id             BIGSERIAL   PRIMARY KEY,
    repository_id  BIGINT      NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    requested_by   BIGINT      REFERENCES users(id) ON DELETE SET NULL,
    status         job_status  NOT NULL DEFAULT 'queued',
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    started_at     TIMESTAMPTZ,
    finished_at    TIMESTAMPTZ,
    worker_id      TEXT,
    error          TEXT,
    stats          JSONB,
    CHECK (started_at IS NULL OR started_at >= created_at),
    CHECK (finished_at IS NULL OR finished_at >= COALESCE(started_at, created_at)),
    CHECK (status NOT IN ('queued', 'running') OR finished_at IS NULL),
    CHECK (status <> 'running' OR started_at IS NOT NULL)
);
-- at most one queued-or-running job per repository (partial unique index)
CREATE UNIQUE INDEX analysis_jobs_one_active_uq
    ON analysis_jobs (repository_id) WHERE status IN ('queued', 'running');
CREATE INDEX analysis_jobs_queue_idx ON analysis_jobs (created_at) WHERE status = 'queued';
CREATE INDEX analysis_jobs_repo_idx  ON analysis_jobs (repository_id, created_at DESC);
```

Mined repository data

```
CREATE TABLE developers (
    id             BIGSERIAL PRIMARY KEY,
    repository_id  BIGINT NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    email          TEXT   NOT NULL,
    name           TEXT   NOT NULL,
    UNIQUE (repository_id, email)
);

CREATE TABLE commits (
    id             BIGSERIAL   PRIMARY KEY,
    repository_id  BIGINT      NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    sha            CHAR(40)    NOT NULL CHECK (sha ~ '^[0-9a-f]{40}$'),
    author_id      BIGINT      NOT NULL REFERENCES developers(id) ON DELETE CASCADE,
    authored_at    TIMESTAMPTZ NOT NULL,
    committed_at   TIMESTAMPTZ NOT NULL,
    parent_count   SMALLINT    NOT NULL CHECK (parent_count >= 0),
    is_merge       BOOLEAN     GENERATED ALWAYS AS (parent_count > 1) STORED,
    subject        TEXT        NOT NULL,
    body           TEXT        NOT NULL DEFAULT '',
    files_changed  INTEGER     NOT NULL DEFAULT 0 CHECK (files_changed >= 0),
    insertions     INTEGER     NOT NULL DEFAULT 0 CHECK (insertions >= 0),
    deletions      INTEGER     NOT NULL DEFAULT 0 CHECK (deletions >= 0),
    UNIQUE (repository_id, sha)
);
CREATE INDEX commits_repo_time_idx   ON commits (repository_id, authored_at DESC);
CREATE INDEX commits_message_fts_idx ON commits
    USING GIN (to_tsvector('english', subject || ' ' || body));

CREATE TABLE files (
    id             BIGSERIAL PRIMARY KEY,
    repository_id  BIGINT  NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    path           TEXT    NOT NULL,
    exists_at_head BOOLEAN NOT NULL DEFAULT FALSE,
    UNIQUE (repository_id, path)
);

CREATE TABLE file_changes (
    commit_id    BIGINT      NOT NULL REFERENCES commits(id) ON DELETE CASCADE,
    file_id      BIGINT      NOT NULL REFERENCES files(id)   ON DELETE CASCADE,
    change       change_kind NOT NULL,
    old_path     TEXT,
    insertions   INTEGER     CHECK (insertions >= 0),
    deletions    INTEGER     CHECK (deletions >= 0),
    PRIMARY KEY (commit_id, file_id),
```

```
    CHECK (change NOT IN ('R', 'C') OR old_path IS NOT NULL)
);
CREATE INDEX file_changes_file_idx ON file_changes (file_id);

CREATE TABLE hunks (
    id         BIGSERIAL PRIMARY KEY,
    commit_id  BIGINT  NOT NULL REFERENCES commits(id) ON DELETE CASCADE,
    file_id    BIGINT  NOT NULL REFERENCES files(id)   ON DELETE CASCADE,
    old_start  INTEGER NOT NULL CHECK (old_start >= 0),
    old_lines  INTEGER NOT NULL CHECK (old_lines >= 0),
    new_start  INTEGER NOT NULL CHECK (new_start >= 0),
    new_lines  INTEGER NOT NULL CHECK (new_lines >= 0)
);

CREATE TABLE coupling_edges (
    repository_id  BIGINT  NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    file_a_id      BIGINT  NOT NULL REFERENCES files(id) ON DELETE CASCADE,
    file_b_id      BIGINT  NOT NULL REFERENCES files(id) ON DELETE CASCADE,
    cochanges      INTEGER NOT NULL CHECK (cochanges > 0),
    confidence_ab  REAL    NOT NULL CHECK (confidence_ab BETWEEN 0 AND 1),
    confidence_ba  REAL    NOT NULL CHECK (confidence_ba BETWEEN 0 AND 1),
    jaccard        REAL    NOT NULL CHECK (jaccard BETWEEN 0 AND 1),
    PRIMARY KEY (file_a_id, file_b_id),
    CHECK (file_a_id < file_b_id)
);
CREATE INDEX coupling_repo_strength_idx ON coupling_edges (repository_id, jaccard DESC);

CREATE TABLE file_ownership (
    file_id       BIGINT  NOT NULL REFERENCES files(id)      ON DELETE CASCADE,
    developer_id  BIGINT  NOT NULL REFERENCES developers(id) ON DELETE CASCADE,
    lines         INTEGER NOT NULL CHECK (lines > 0),
    PRIMARY KEY (file_id, developer_id)
);

CREATE TABLE commit_embeddings (
    commit_id  BIGINT      PRIMARY KEY REFERENCES commits(id) ON DELETE CASCADE,
    embedding  vector(256) NOT NULL
);
CREATE INDEX commit_embeddings_hnsw_idx
    ON commit_embeddings USING hnsw (embedding vector_cosine_ops);

CREATE TABLE audit_log (
    id           BIGSERIAL   PRIMARY KEY,
    user_id      BIGINT      REFERENCES users(id) ON DELETE SET NULL,
    action       TEXT        NOT NULL,
    target_type  TEXT,
    target_id    BIGINT,
    details      JSONB       NOT NULL DEFAULT '{}'::jsonb,
    at           TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX audit_log_at_idx ON audit_log (at DESC);
```

5.3 Schema evolution – migration 002 (ALTER / DROP / CREATE)

```
-- generated column flags bot accounts
ALTER TABLE developers
    ADD COLUMN is_bot BOOLEAN GENERATED ALWAYS AS (
        email ~* '(\[bot\]@|^bot@|dependabot|renovate|github-actions)'
        OR name ~* '\[bot\]$'
    ) STORED;


ALTER TABLE audit_log
    ADD CONSTRAINT audit_log_action_format CHECK (action ~ '^[a-z][a-z_]{1,62}$');


-- replace a single-column index with a composite one
DROP INDEX commits_author_idx;
```

```
CREATE INDEX commits_author_time_idx ON commits (author_id, authored_at DESC);
```

5.4 DML – INSERT, UPDATE, DELETE

INSERT

```
-- register a user (race-free: duplicate e-mail returns no row)
INSERT INTO users (email, display_name, password_hash, role)
VALUES ($1, $2, $3, 'analyst')
ON CONFLICT DO NOTHING
RETURNING id;

-- create a session (only the SHA-256 hash of the token is stored)
INSERT INTO sessions (token_hash, user_id, expires_at)
VALUES ($1, $2, now() + make_interval(hours => $3));

-- register a repository and queue its first analysis (one transaction)
INSERT INTO repositories (name, source_type, source, branch, created_by)
VALUES ($1, $2, $3, $4, $5) RETURNING id;
INSERT INTO analysis_jobs (repository_id, requested_by) VALUES ($1, $2);

-- queue re-analysis only if none is active
INSERT INTO analysis_jobs (repository_id, requested_by) VALUES ($1, $2)
ON CONFLICT (repository_id) WHERE status IN ('queued', 'running') DO NOTHING
RETURNING id;

-- bulk load mined developers with unnest
INSERT INTO developers (repository_id, email, name)
SELECT $1, e, n FROM unnest($2::text[], $3::text[]) AS t(e, n)
RETURNING id, email;
```

UPDATE

```
-- cancel an active job
UPDATE analysis_jobs SET status = 'cancelled', finished_at = now()
 WHERE id = $1 AND status IN ('queued', 'running')
RETURNING repository_id;

-- change role / active flag of a user
UPDATE users SET role = COALESCE($2, role), is_active = COALESCE($3, is_active)
 WHERE id = $1 RETURNING id;

-- remember the analysed commit
UPDATE repositories SET head_sha = $2 WHERE id = $1;
```

DELETE

```
DELETE FROM sessions WHERE token_hash = $1;                 -- sign out
DELETE FROM sessions WHERE user_id = $1;                    -- deactivated user
DELETE FROM repositories WHERE id = $1;                     -- cascades to all mined data
DELETE FROM coupling_edges WHERE repository_id = $1;        -- before re-analysis
DELETE FROM developers WHERE repository_id = $1;
DELETE FROM files WHERE repository_id = $1;
```

5.5 Queries

Q1. Claim the next job (subquery, row locking)

```
UPDATE analysis_jobs SET status = 'running', started_at = now(), worker_id = $1
 WHERE id = (SELECT id FROM analysis_jobs
              WHERE status = 'queued'
              ORDER BY created_at
              FOR UPDATE SKIP LOCKED
              LIMIT 1)
RETURNING id, repository_id;
```

Q2. Core contributors (CTE, GROUP BY, HAVING, subquery)

```
WITH human AS (
  SELECT c.author_id FROM commits c JOIN developers d ON d.id = c.author_id
   WHERE c.repository_id = $1 AND NOT d.is_bot
), average AS (
  SELECT avg(n)::float AS avg_commits
    FROM (SELECT count(*) AS n FROM human GROUP BY author_id) per_dev
)
SELECT d.id, d.name, d.email,
       count(*)                                        AS commits,
       count(*)::float / (SELECT count(*) FROM human) AS share,
       a.avg_commits
  FROM human h
  JOIN developers d ON d.id = h.author_id
 CROSS JOIN average a
 GROUP BY d.id, a.avg_commits
HAVING count(*) >= a.avg_commits
 ORDER BY commits DESC, d.name;
```

Q3. Shared files (3-way JOIN, GROUP BY, HAVING)

```
SELECT f.id, f.path,
       count(DISTINCT c.author_id)                    AS authors,
       count(*)                                       AS revisions,
       COALESCE(sum(fc.insertions + fc.deletions), 0) AS churn
  FROM file_changes fc
  JOIN commits c    ON c.id = fc.commit_id
  JOIN developers d ON d.id = c.author_id
  JOIN files f      ON f.id = fc.file_id
 WHERE f.repository_id = $1 AND f.exists_at_head AND NOT c.is_merge AND NOT d.is_bot
 GROUP BY f.id
HAVING count(DISTINCT c.author_id) >= $2
 ORDER BY authors DESC, revisions DESC, f.path
 LIMIT 100;
```

Q4. Hotspots (view + correlated subquery)

```
SELECT h.file_id, h.path, h.revisions, h.authors, h.churn, h.last_changed_at,
       (SELECT count(*) FROM coupling_edges e
         WHERE e.file_a_id = h.file_id OR e.file_b_id = h.file_id) AS coupled_files
  FROM file_hotspots h
 WHERE h.repository_id = $1 AND h.exists_at_head
 ORDER BY h.revisions DESC, h.churn DESC, h.path
 LIMIT $2;
```

Q5. Change coupling (join through coupling_edges)

```
SELECT e.file_a_id, a.path AS file_a, e.file_b_id, b.path AS file_b,
       e.cochanges, e.confidence_ab, e.confidence_ba, e.jaccard
  FROM coupling_edges e
  JOIN files a ON a.id = e.file_a_id
  JOIN files b ON b.id = e.file_b_id
 WHERE e.repository_id = $1 AND e.jaccard >= $2
   AND a.exists_at_head AND b.exists_at_head
 ORDER BY e.jaccard DESC, e.cochanges DESC
 LIMIT $3;
```

Q6. Full-text search (GIN index, ranking, highlighting)

```
SELECT c.sha, c.subject,
       ts_headline('english', c.subject || ' ' || c.body, q) AS headline,
       ts_rank(to_tsvector('english', c.subject || ' ' || c.body), q) AS score
  FROM commits c, websearch_to_tsquery('english', $2) q
 WHERE c.repository_id = $1
   AND to_tsvector('english', c.subject || ' ' || c.body) @@ q
 ORDER BY score DESC;
```

5.6 Views and materialised view

```
CREATE VIEW developer_activity AS
```

```
SELECT d.repository_id, d.id AS developer_id, d.name, d.email,
       count(c.id)                           AS commits,
       count(c.id) FILTER (WHERE c.is_merge) AS merges,
       COALESCE(sum(c.insertions), 0)        AS insertions,
       COALESCE(sum(c.deletions), 0)         AS deletions,
       min(c.authored_at)                    AS first_commit_at,
       max(c.authored_at)                    AS last_commit_at,
       count(DISTINCT c.authored_at::date)   AS active_days
FROM developers d LEFT JOIN commits c ON c.author_id = d.id
GROUP BY d.repository_id, d.id;

-- redefined in migration 002 to ignore bots
CREATE VIEW file_hotspots AS
SELECT f.repository_id, f.id AS file_id, f.path, f.exists_at_head,
       count(fc.commit_id)                                    AS revisions,
       count(DISTINCT c.author_id) FILTER (WHERE NOT d.is_bot) AS authors,
       COALESCE(sum(fc.insertions + fc.deletions), 0)         AS churn,
       max(c.authored_at)                                     AS last_changed_at
FROM files f
JOIN file_changes fc ON fc.file_id = f.id
JOIN commits c       ON c.id = fc.commit_id
JOIN developers d    ON d.id = c.author_id
GROUP BY f.id;

CREATE MATERIALIZED VIEW monthly_activity AS
SELECT c.repository_id,
       date_trunc('month', c.authored_at)::date AS month,
       count(*) AS commits, count(DISTINCT c.author_id) AS authors,
       sum(c.insertions) AS insertions, sum(c.deletions) AS deletions
FROM commits c GROUP BY 1, 2;
CREATE UNIQUE INDEX monthly_activity_uq ON monthly_activity (repository_id, month);

REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_activity;   -- after each analysis
```

5.7 Stored function, stored procedure and triggers

Function: bus_factor() (window functions)

```
CREATE FUNCTION bus_factor(p_repository_id BIGINT) RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    WITH per_dev AS (
        SELECT o.developer_id, sum(o.lines) AS lines
        FROM file_ownership o JOIN files f ON f.id = o.file_id
        WHERE f.repository_id = p_repository_id
        GROUP BY o.developer_id
    ), ranked AS (
        SELECT lines,
               sum(lines) OVER (ORDER BY lines DESC, developer_id
                                ROWS UNBOUNDED PRECEDING) AS running,
               sum(lines) OVER () AS total
        FROM per_dev
    )
    SELECT count(*)::int FROM ranked WHERE running - lines <= total / 2.0
$$;

SELECT bus_factor(1);
```

Procedure: purge_expired_sessions()

```
CREATE PROCEDURE purge_expired_sessions(INOUT removed INTEGER DEFAULT NULL)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM sessions WHERE expires_at < now();
    GET DIAGNOSTICS removed = ROW_COUNT;
END $$;

CALL purge_expired_sessions();     -- executed on every sign-in
```

Triggers

```
CREATE FUNCTION touch_updated_at() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;
CREATE TRIGGER repositories_touch BEFORE UPDATE ON repositories
    FOR EACH ROW EXECUTE FUNCTION touch_updated_at();

-- the system must always keep one active administrator
CREATE FUNCTION protect_last_admin() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.role = 'admin' AND OLD.is_active
       AND (TG_OP = 'DELETE' OR NEW.role <> 'admin' OR NOT NEW.is_active)
       AND NOT EXISTS (SELECT 1 FROM users
                       WHERE role = 'admin' AND is_active AND id <> OLD.id) THEN
        RAISE EXCEPTION 'cannot remove or demote the last active administrator'
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN COALESCE(NEW, OLD);
END $$;
CREATE TRIGGER users_protect_last_admin BEFORE UPDATE OR DELETE ON users
    FOR EACH ROW EXECUTE FUNCTION protect_last_admin();

-- stamp the repository when a job succeeds
CREATE FUNCTION stamp_repository_analyzed() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status = 'succeeded' AND OLD.status IS DISTINCT FROM 'succeeded' THEN
        UPDATE repositories SET last_analyzed_at = NEW.finished_at
         WHERE id = NEW.repository_id;
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER analysis_jobs_stamp AFTER UPDATE OF status ON analysis_jobs
    FOR EACH ROW EXECUTE FUNCTION stamp_repository_analyzed();
```

5.8 Transactions and concurrency

- Each analysis deletes old results and bulk-inserts new ones in one transaction, so readers see either the old or the new data, never a mix.
- Repository registration, job creation and the audit record are committed together.
- FOR UPDATE SKIP LOCKED lets several workers poll the queue without blocking or double-processing a job.
- The partial unique index analysis_jobs_one_active_uq guarantees at most one queued/running job per repository.

6. NoSQL / VECTOR DATABASE COMMANDS

6.1 Vector database – pgvector

Each commit message is converted into a 256-dimension unit vector by feature hashing (words and adjacent word pairs are hashed into buckets with a pseudo-random sign, then L2-normalised). The vectors are stored in a vector(256) column and searched by cosine distance using an HNSW index. This is deterministic and needs no external model or API.

```
CREATE EXTENSION IF NOT EXISTS vector;


CREATE TABLE commit_embeddings (
    commit_id  BIGINT      PRIMARY KEY REFERENCES commits(id) ON DELETE CASCADE,
    embedding  vector(256) NOT NULL
);
CREATE INDEX commit_embeddings_hnsw_idx
    ON commit_embeddings USING hnsw (embedding vector_cosine_ops);


-- store an embedding (vector literal built by toPgVector())
INSERT INTO commit_embeddings (commit_id, embedding) VALUES ($1, '[0.012,-0.044, ...]'::vector);


-- "commits like this text": nearest neighbours by cosine distance (<=>)
SELECT c.sha, c.subject, (1 - (e.embedding <=> $2::vector)) AS score
  FROM commit_embeddings e
  JOIN commits c    ON c.id = e.commit_id
  JOIN developers d ON d.id = c.author_id
 WHERE c.repository_id = $1
 ORDER BY e.embedding <=> $2::vector
 LIMIT $3;
```

6.2 Graph database – Neo4j (Cypher)

PostgreSQL remains the source of truth; after each analysis the repository is projected into Neo4j. The graph can always be rebuilt from PostgreSQL.

```
(:Repository)<-[:IN_REPO]-(:Developer)-[:TOUCHED {commits, churn}]->(:File)-[:IN_REPO]-
>(:Repository)
(:Developer)-[:OWNS {lines}]->(:File)
(:File)-[:COUPLED_WITH {cochanges, jaccard}]-(:File)
```

Constraints and indexes (npm run neo4j:init)

```
CREATE CONSTRAINT repository_id IF NOT EXISTS FOR (r:Repository) REQUIRE r.id IS UNIQUE;
CREATE CONSTRAINT developer_id  IF NOT EXISTS FOR (d:Developer)  REQUIRE d.id IS UNIQUE;
CREATE CONSTRAINT file_id       IF NOT EXISTS FOR (f:File)       REQUIRE f.id IS UNIQUE;
CREATE INDEX developer_repo IF NOT EXISTS FOR (d:Developer) ON (d.repoId);
CREATE INDEX file_repo      IF NOT EXISTS FOR (f:File)      ON (f.repoId);
```

Building the projection

```
MERGE (r:Repository {id: $repoId}) SET r.name = $name;


UNWIND $rows AS row
MATCH (r:Repository {id: $repoId})
CREATE (d:Developer {id: row.id, repoId: $repoId, name: row.name, email: row.email})-[:IN_REPO]-
>(r);


UNWIND $rows AS row
MATCH (r:Repository {id: $repoId})
CREATE (f:File {id: row.id, repoId: $repoId, path: row.path, alive: row.alive})-[:IN_REPO]->(r);


UNWIND $rows AS row
```

```
MATCH (d:Developer {id: row.dev}), (f:File {id: row.file})
CREATE (d)-[:TOUCHED {commits: row.commits, churn: row.churn}]->(f);

UNWIND $rows AS row
MATCH (d:Developer {id: row.dev}), (f:File {id: row.file})
CREATE (d)-[:OWNS {lines: row.lines}]->(f);

UNWIND $rows AS row
MATCH (a:File {id: row.a}), (b:File {id: row.b})
CREATE (a)-[:COUPLED_WITH {cochanges: row.cochanges, jaccard: row.jaccard}]->(b);

// clear the previous projection in batches
MATCH (n) WHERE (n:Developer OR n:File) AND n.repoId = $repoId
WITH n LIMIT 5000 DETACH DELETE n RETURN count(*) AS deleted;
```

Reviewer recommendation

```
MATCH (f:File {id: $fileId})
OPTIONAL MATCH (f)-[c:COUPLED_WITH]-(g:File)
WITH f, collect({file: g, w: c.jaccard}) AS neighbours
UNWIND [{file: f, w: 1.0}] + [n IN neighbours WHERE n.file IS NOT NULL] AS target
WITH target.file AS file, target.w AS w
MATCH (d:Developer)-[t:TOUCHED]->(file)
WITH d, sum(t.commits * w) AS score, collect(DISTINCT file.path)[0..5] AS via
RETURN d.name AS developer, d.email AS email, round(score * 100) / 100.0 AS score, via
ORDER BY score DESC LIMIT 8;
```

Collaboration pairs

```
MATCH (a:Developer {repoId: $repoId})-[:TOUCHED]->(f:File)<-[:TOUCHED]-(b:Developer)
WHERE a.id < b.id
RETURN a.name AS a, b.name AS b, count(DISTINCT f) AS shared_files
ORDER BY shared_files DESC LIMIT 30;
```

Knowledge silos (one developer owns ≥ 90% of a file)

```
MATCH (f:File {repoId: $repoId, alive: true})<-[o:OWNS]-(d:Developer)
WITH f, sum(o.lines) AS total, collect({d: d, lines: o.lines}) AS owners
UNWIND owners AS ow
WITH f, total, ow WHERE total >= 5 AND toFloat(ow.lines) / total >= 0.9
RETURN f.path AS path, ow.d.name AS owner, round(100.0 * ow.lines / total) / 100.0 AS share
ORDER BY total DESC LIMIT 25;
```

6.3 Key-value store – Redis

Redis is an accelerator, not a source of truth. Every call is wrapped so the application falls back to PostgreSQL if Redis is unavailable.

Command	Use in the application
GET cache:repo:<id>:<report>	Read a cached report
SET cache:repo:<id>:<report> <json> EX 600	Cache a computed report for 10 minutes
SCAN 0 MATCH cache:repo:<id>:* COUNT 200 DEL <keys…>	Invalidate a repository's cache after re-analysis
SET job:<jobId>:progress <json> EX 86400	Store live job progress (stage, percent, message)
GET job:<jobId>:progress	Read progress for the job panel
INCR rl:<bucket>:<window> EXPIRE rl:<bucket>:<window> <seconds>	Fixed-window rate limiting for login / registration

7. NEXT.JS – PROJECT TREE STRUCTURE

```
git-repository-forensics/
├── app/                                # Next.js App Router
│   ├── layout.tsx                      # root layout
│   ├── page.tsx                        # landing / redirect
│   ├── globals.css
│   ├── login/page.tsx                  # sign-in page
│   ├── register/page.tsx               # self-registration page
│   ├── (app)/                          # authenticated area
│   │   ├── layout.tsx                  # navigation shell
│   │   ├── admin/page.tsx              # users, roles, audit log
│   │   ├── repositories/
│   │   │   ├── page.tsx                # repository list + registration
│   │   │   └── [id]/
│   │   │       ├── page.tsx            # dashboard: overview, developers, hotspots,
│   │   │       │                       #   coupling, team, search, graph, jobs
│   │   │       └── files/[fileId]/page.tsx   # file detail + suggested reviewers
│   └── api/                            # JSON API route handlers
│       ├── auth/{login,logout,register}/route.ts
│       ├── repositories/route.ts
│       ├── repositories/[id]/route.ts
│       ├── repositories/[id]/analyze/route.ts
│       ├── repositories/[id]/report/route.ts
│       ├── jobs/[id]/route.ts
│       ├── jobs/[id]/cancel/route.ts
│       └── admin/users/[id]/route.ts
├── components/                         # React components
│   ├── Actions.tsx   ActivityChart.tsx   AuthForm.tsx
│   ├── JobPanel.tsx  client.ts
├── lib/                                # server-side logic
│   ├── analysis/
│   │   ├── pipeline.ts                 # mining + persistence pipeline
│   │   ├── coupling.ts                 # change-coupling metrics
│   │   └── embedding.ts                # commit-message embeddings
│   ├── git/{run.ts, parse.ts}          # safe git execution + output parsing
│   ├── graph/projection.ts             # PostgreSQL -> Neo4j projection
│   ├── queries.ts                      # all report queries (SQL + Cypher)
│   ├── db.ts  redis.ts  neo4j.ts       # database clients
│   ├── auth.ts  api.ts  env.ts  format.ts  migrate.ts  repo-source.ts
├── workers/
│   └── analysis-worker.ts              # background job worker
├── db/migrations/
│   ├── 001_schema.sql
│   └── 002_team_analytics.sql
├── scripts/                            # migrate, seed, reset, neo4j-init, fixture repo
├── tests/
│   ├── unit/{analysis,git-parse,repo-source}.test.ts
│   └── integration/pipeline.test.ts
├── docs/                               # architecture, API, database design, security, report
├── docker-compose.yml                  # PostgreSQL+pgvector, Neo4j, Redis
├── Makefile   .env.example
├── package.json  tsconfig.json  next.config.mjs  tailwind.config.ts
└── README.md
```

Role of the main folders

Folder / file	Role
app/	Pages and API route handlers (UI and backend in one Next.js project)
components/	Reusable client components: charts, forms, job-progress panel
lib/queries.ts	Every SQL and Cypher query used by the reports, with Redis caching
lib/analysis/pipeline.ts	Mines Git, writes results in one transaction, refreshes views, projects to Neo4j
workers/analysis-worker.ts	Polls for queued jobs and runs the pipeline

Folder / file	Role
db/migrations/	Versioned SQL schema
tests/	58 unit and integration tests (Vitest)

8. SAMPLE SCREENSHOTS – APPLICATION CODE AND DEMO

8.1 Application code

db/migrations/001_schema.sql — commits table (generated column, constraints, GIN index)

```
);
CREATE INDEX commits_repo_time_idx   ON commits (repository_id, authored_at DESC);
CREATE INDEX commits_author_idx      ON commits (author_id);
-- Full-text search over commit messages.
CREATE INDEX commits_message_fts_idx ON commits
    USING GIN (to_tsvector('english', subject || ' ' || body));

CREATE TABLE files (
    id            BIGSERIAL PRIMARY KEY,
    repository_id  BIGINT    NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    path           TEXT      NOT NULL,
    exists_at_head BOOLEAN   NOT NULL DEFAULT FALSE,
    UNIQUE (repository_id, path)
);

CREATE TABLE file_changes (
    commit_id    BIGINT      NOT NULL REFERENCES commits(id) ON DELETE CASCADE,
    file_id      BIGINT      NOT NULL REFERENCES files(id)   ON DELETE CASCADE,
    change       change_kind NOT NULL,
    old_path     TEXT,
    -- NULL for binary files, where git reports no line counts
    insertions   INTEGER     CHECK (insertions >= 0),
```

Figure 1: Relational schema – commits table with generated column, CHECK constraints and GIN full-text index

db/migrations/001_schema.sql — bus_factor() stored function

```
CREATE UNIQUE INDEX monthly_activity_uq ON monthly_activity (repository_id, month);

-- Bus factor: the smallest number of developers who together own more than
-- half of the blamed lines at HEAD.
CREATE FUNCTION bus_factor(p_repository_id BIGINT) RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    WITH per_dev AS (
        SELECT o.developer_id, sum(o.lines) AS lines
        FROM file_ownership o
        JOIN files f ON f.id = o.file_id
        WHERE f.repository_id = p_repository_id
        GROUP BY o.developer_id
    ), ranked AS (
        SELECT lines,
               sum(lines) OVER (ORDER BY lines DESC, developer_id
                                ROWS UNBOUNDED PRECEDING) AS running,
               sum(lines) OVER ()                         AS total
        FROM per_dev
    )
    -- a developer is needed while the owners before them hold at most half
    SELECT count(*)::int FROM ranked WHERE running - lines <= total / 2.0
$$;
```

Figure 2: bus_factor() stored function using window functions

lib/queries.ts — full-text search and pgvector similarity search

```ts
export function searchCommits(repoId: number, text: string, limit = 30) {
  return query<CommitRow>(
    `SELECT ${commitCols},
            ts_headline('english', c.subject || ' ' || c.body, q,
                        'StartSel=<<, StopSel=>>, MaxFragments=1, MaxWords=20') AS headline,
            ts_rank(to_tsvector('english', c.subject || ' ' || c.body), q)::float AS score
       FROM commits c JOIN developers d ON d.id = c.author_id,
            websearch_to_tsquery('english', $2) q
      WHERE c.repository_id = $1
        AND to_tsvector('english', c.subject || ' ' || c.body) @@ q
      ORDER BY score DESC, c.authored_at DESC LIMIT $3`,
    [repoId, text, limit],
  );
}

/** Commits whose messages are closest (cosine distance) to free text. */
export function similarCommits(repoId: number, text: string, limit = 15) {
  const v = embed(text);
  if (!v) return Promise.resolve([] as CommitRow[]);
  return query<CommitRow>(
    `SELECT ${commitCols}, (1 - (e.embedding <=> $2::vector))::float AS score
       FROM commit_embeddings e
       JOIN commits c    ON c.id = e.commit_id
       JOIN developers d ON d.id = c.author_id
      WHERE c.repository_id = $1
      ORDER BY e.embedding <=> $2::vector LIMIT $3`,
    [repoId, toPgVector(v), limit],
  );
}
```

Figure 3: Full-text search and pgvector similarity search queries

lib/queries.ts — Neo4j reviewer recommendation (Cypher)

```ts
export type Reviewer = { developer: string; email: string; score: number; via: string[] };

/**
 * Suggests reviewers for a change to a file: people who touched the file or
 * the files most strongly coupled to it, weighted by commits and coupling.
 */
export function suggestReviewers(fileId: number) {
  return cypher<Reviewer>(
    `MATCH (f:File {id: $fileId})
     OPTIONAL MATCH (f)-[c:COUPLED_WITH]-(g:File)
     WITH f, collect({file: g, w: c.jaccard}) AS neighbours
     UNWIND [{file: f, w: 1.0}] + [n IN neighbours WHERE n.file IS NOT NULL] AS target
     WITH target.file AS file, target.w AS w
     MATCH (d:Developer)-[t:TOUCHED]->(file)
     WITH d, sum(t.commits * w) AS score, collect(DISTINCT file.path)[0..5] AS via
     RETURN d.name AS developer, d.email AS email, round(score * 100) / 100.0 AS score, via
     ORDER BY score DESC LIMIT 8`,
    { fileId },
  );
}
```

Figure 4: Neo4j Cypher query for reviewer recommendation

lib/redis.ts — Redis read-through cache

```ts
/**
 * Read-through cache. Any Redis failure falls back to computing the value,
 * so the app keeps working (slower) when Redis is down.
 */
export async function cached<T>(
  key: string,
  ttlSeconds: number,
  compute: () => Promise<T>,
): Promise<T> {
  try {
    const hit = await redis().get(key);
    if (hit !== null) return JSON.parse(hit) as T;
  } catch {
    return compute();
  }
  const value = await compute();
  try {
    await redis().set(key, JSON.stringify(value), "EX", ttlSeconds);
  } catch {
    /* ignore */
  }
```

Figure 5: Redis read-through cache with graceful fallback

lib/analysis/embedding.ts — commit-message embedding

```ts
}

function add(vec: number[], feature: string, weight: number): void {
  const h = fnv1a(feature);
  const sign = (h & 0x80000000) === 0 ? 1 : -1;
  vec[h % DIMENSIONS] += sign * weight;
}

/** Returns a unit vector, or null when the text has no usable words. */
export function embed(text: string): number[] | null {
  const tokens = tokenize(text);
  if (tokens.length === 0) return null;
  const vec = new Array<number>(DIMENSIONS).fill(0);
  for (let i = 0; i < tokens.length; i++) {
    add(vec, tokens[i], 1);
    if (i + 1 < tokens.length) add(vec, `${tokens[i]} ${tokens[i + 1]}`, 0.5);
  }
  const norm = Math.hypot(...vec);
  if (norm === 0) return null;
  return vec.map((v) => v / norm);
}

/** pgvector text literal, e.g. "[0.1,0.2]". */
export function toPgVector(vec: number[]): string {
  return `[${vec.map((v) => (Number.isFinite(v) ? +v.toFixed(6) : 0)).join(",")}]`;
}
```

Figure 6: Commit-message embedding by feature hashing

8.2 Application demo

The screenshots below were captured from the running application after analysing the public repository expressjs/cors (351 commits, 62 developers).

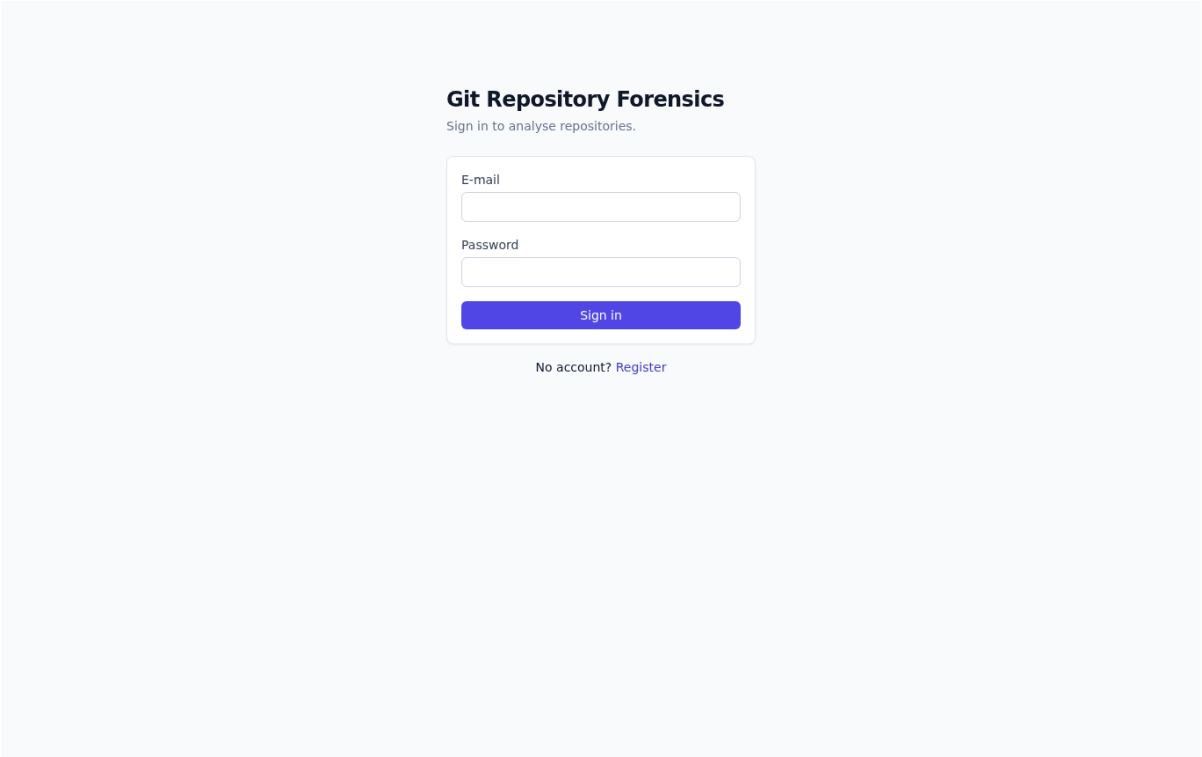


Figure 7: Sign-in page

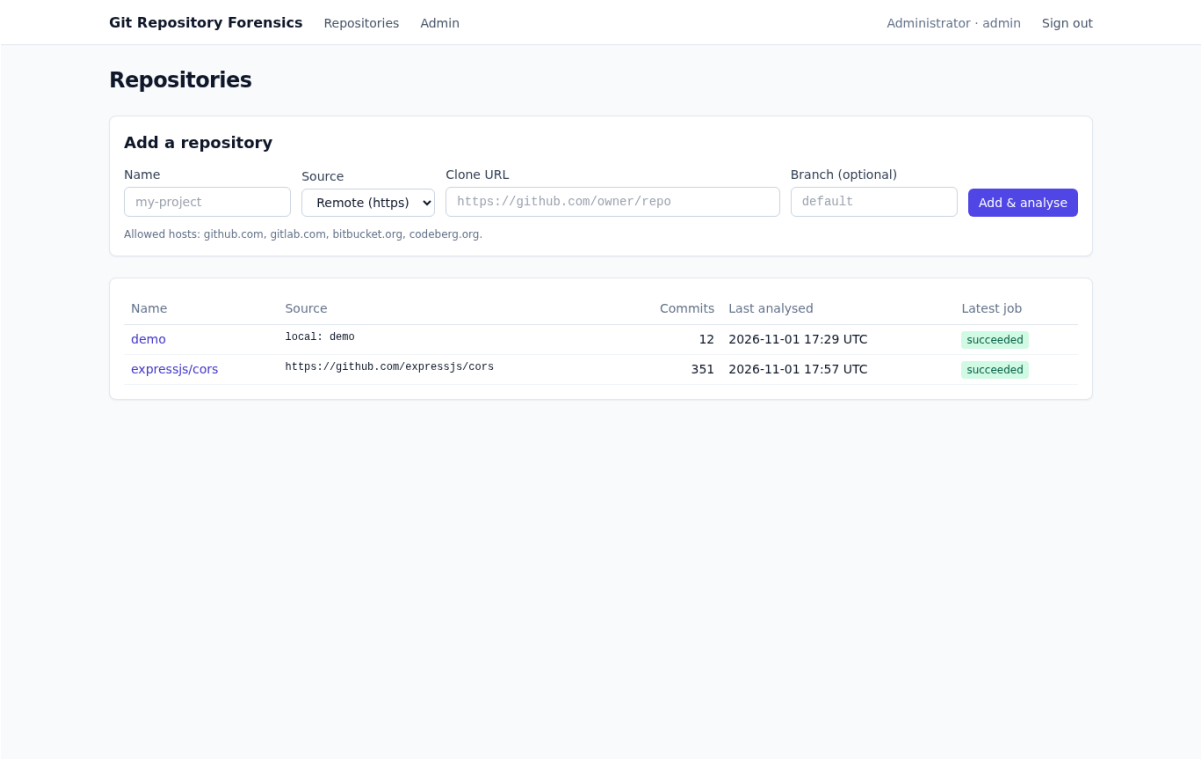


Figure 8: Repositories – register a repository and see analysis status

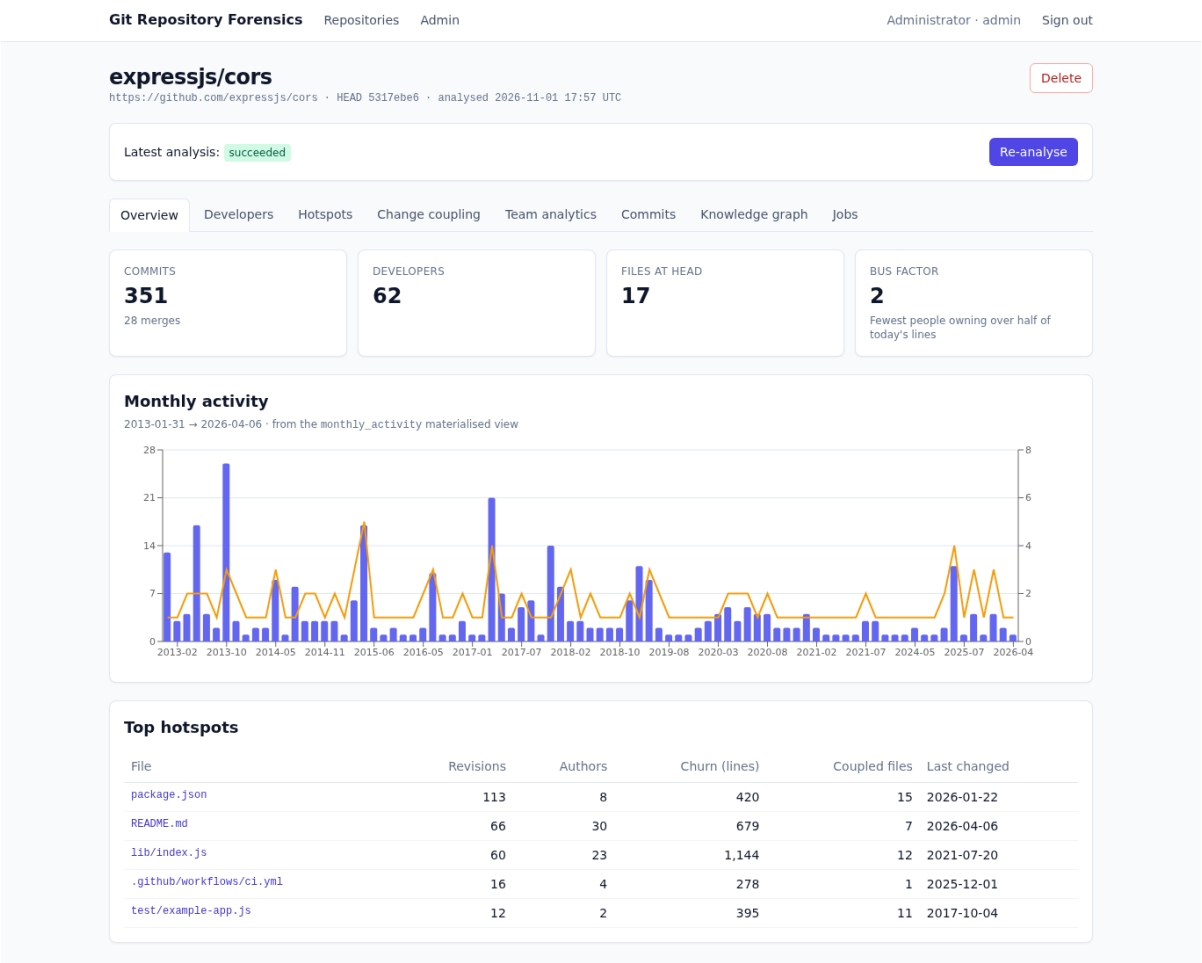


Figure 9: Overview – summary cards, bus factor, monthly activity (materialised view) and top hotspots

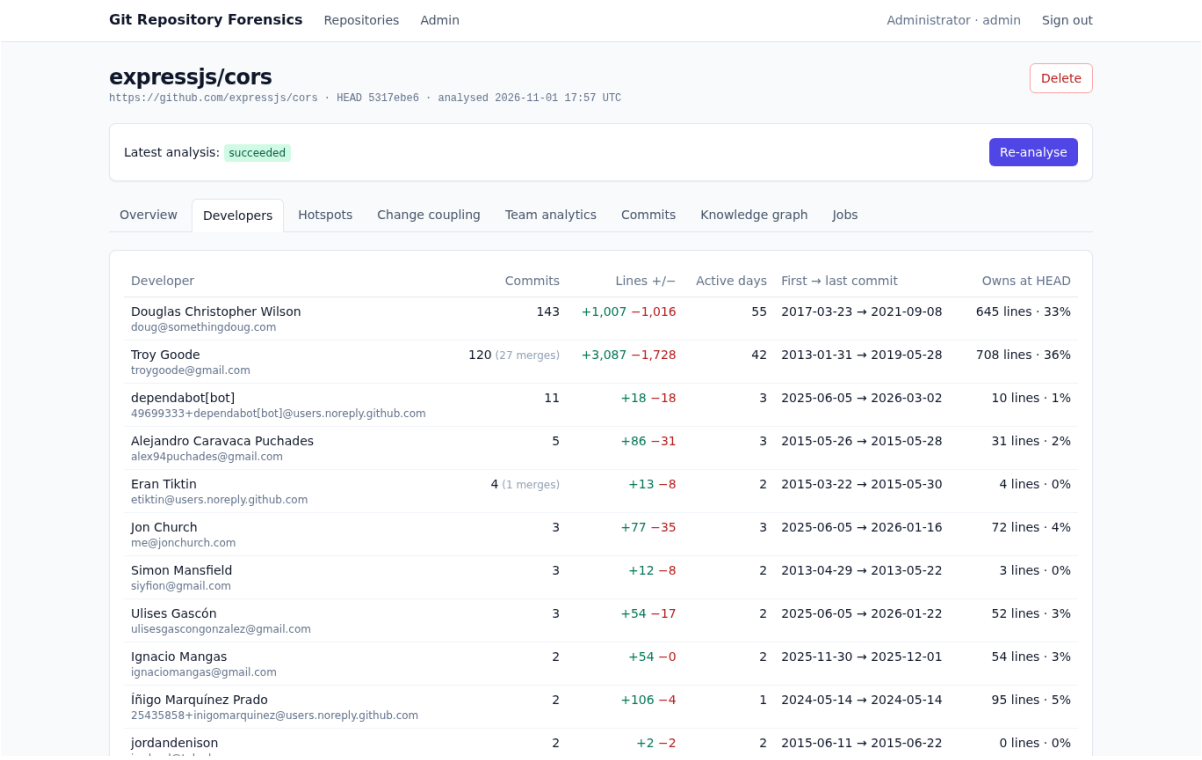


Figure 10: Developers – per-developer activity and ownership

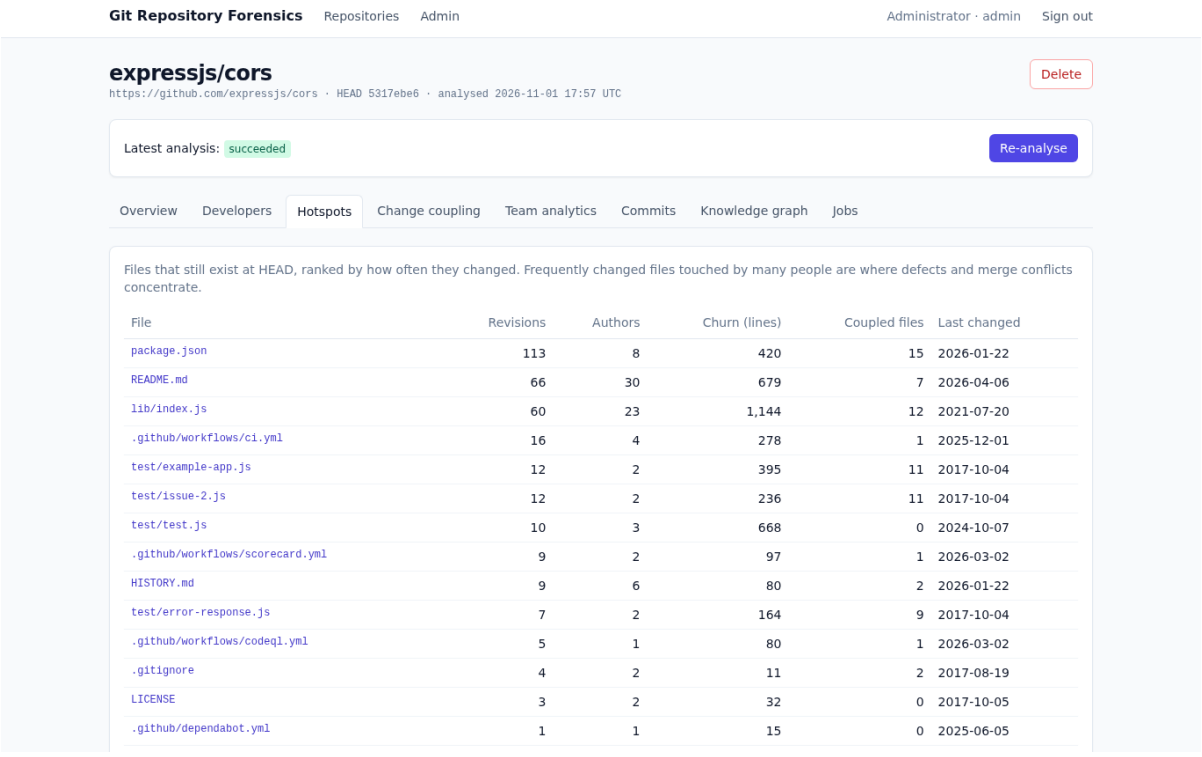


Figure 11: Hotspots – revisions, authors and churn

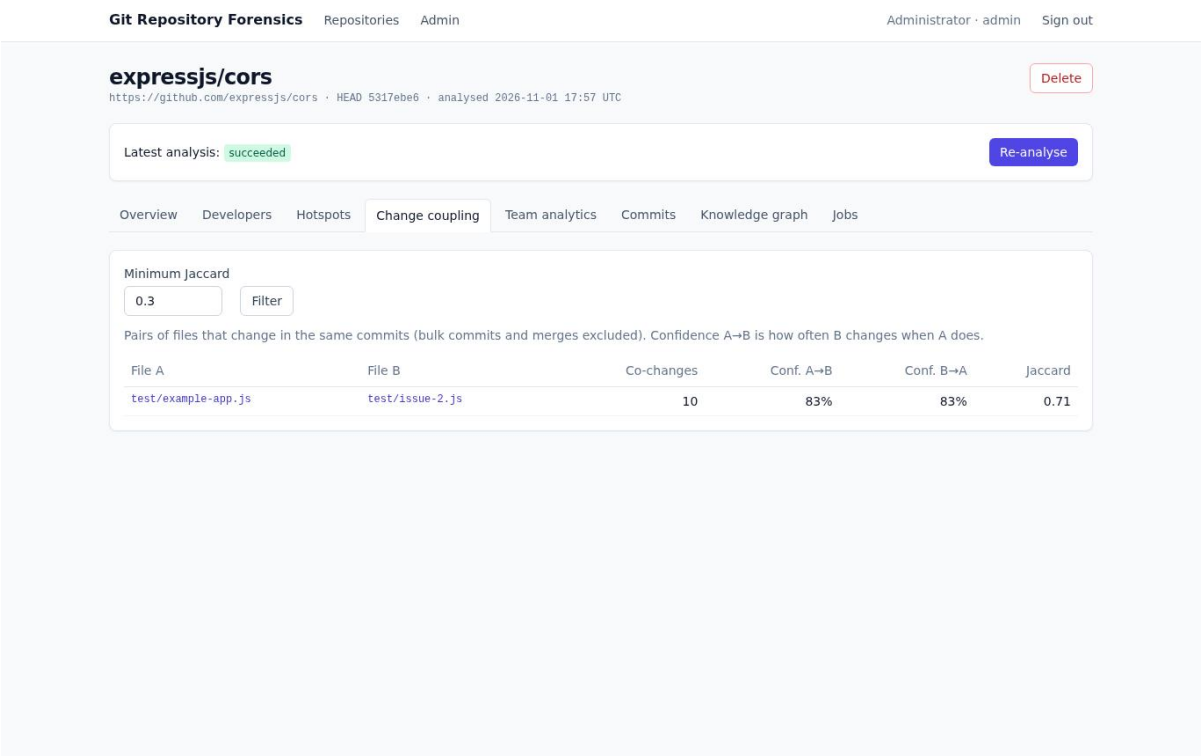


Figure 12: Change coupling filtered by minimum Jaccard similarity

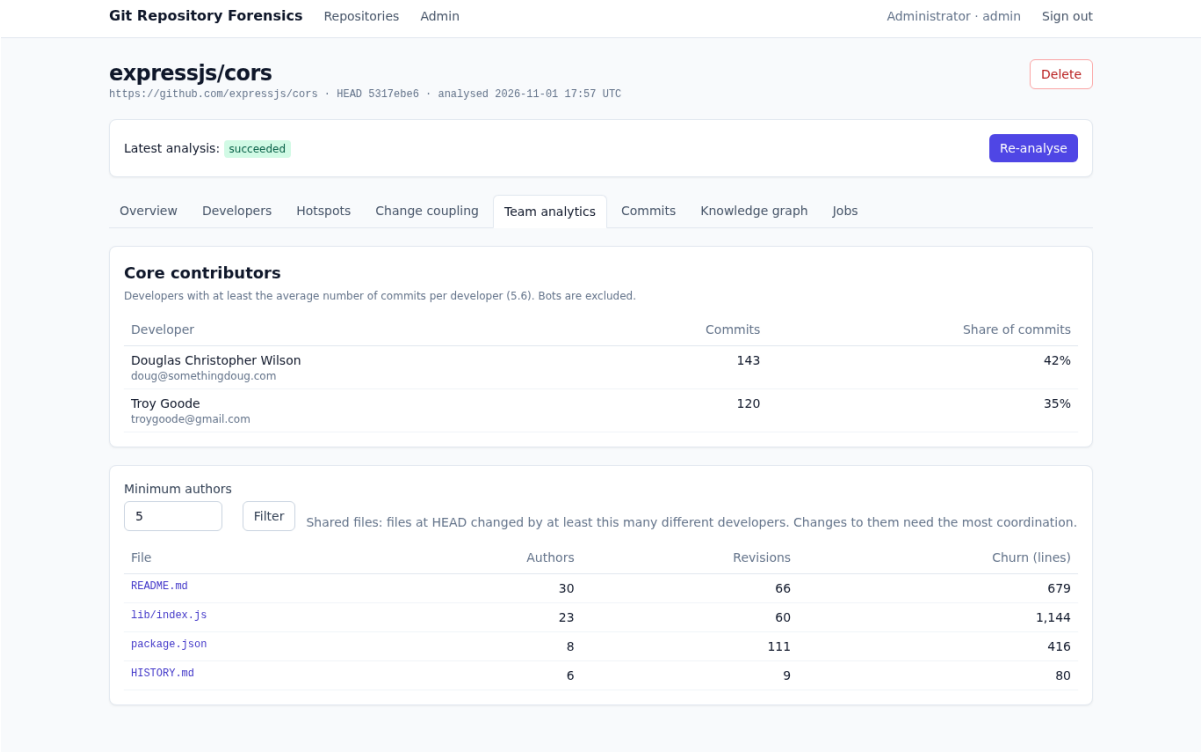


Figure 13: Team analytics – core contributors and shared files (GROUP BY ... HAVING)

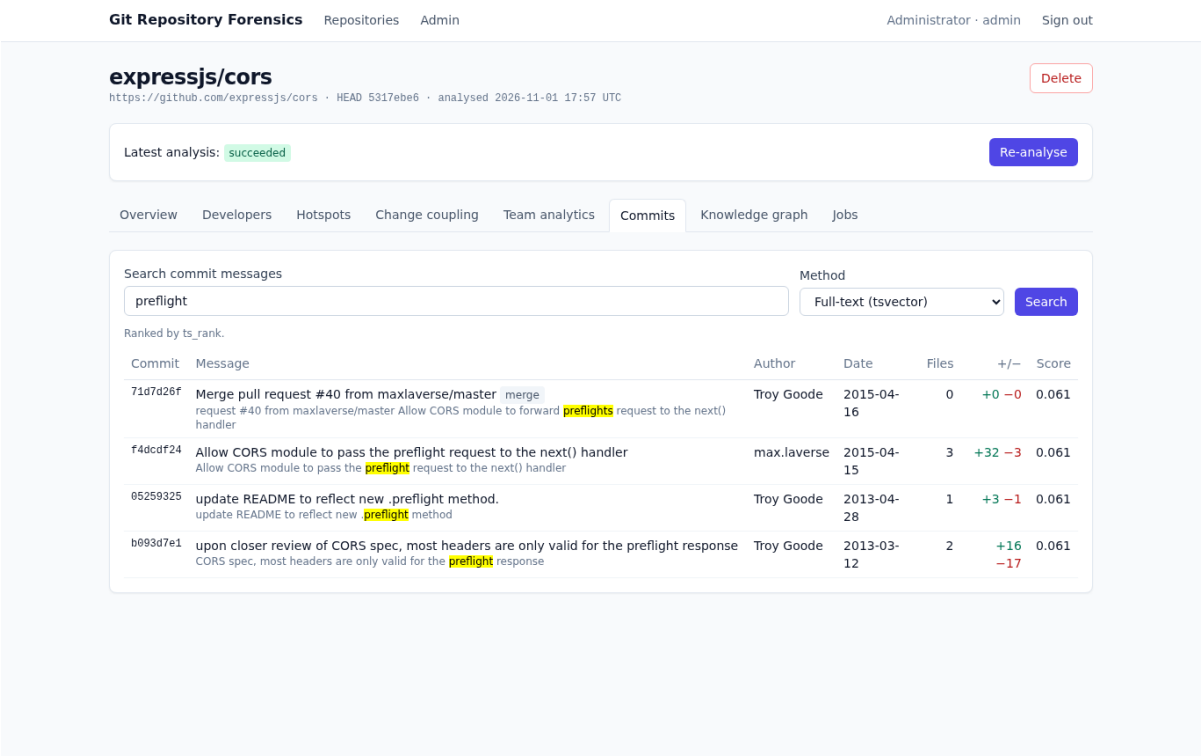


Figure 14: Full-text commit search (tsvector, ranked and highlighted)

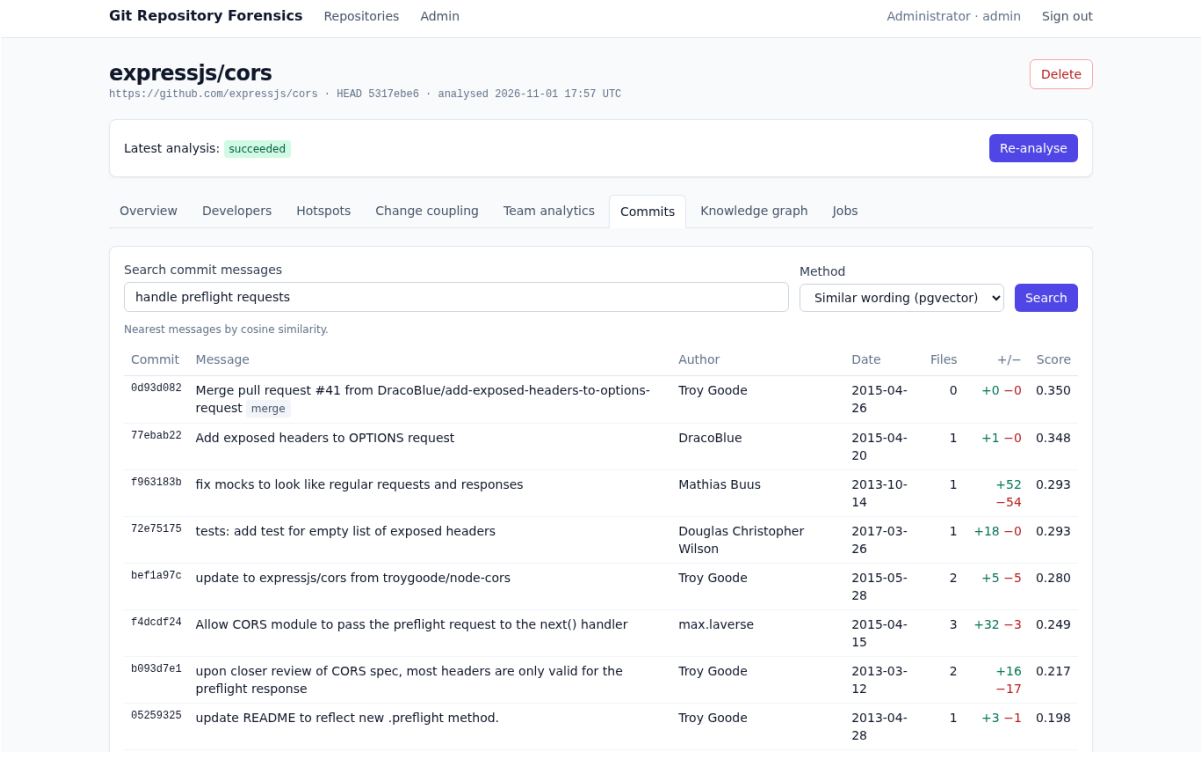


Figure 15: Semantic commit search (pgvector cosine similarity)

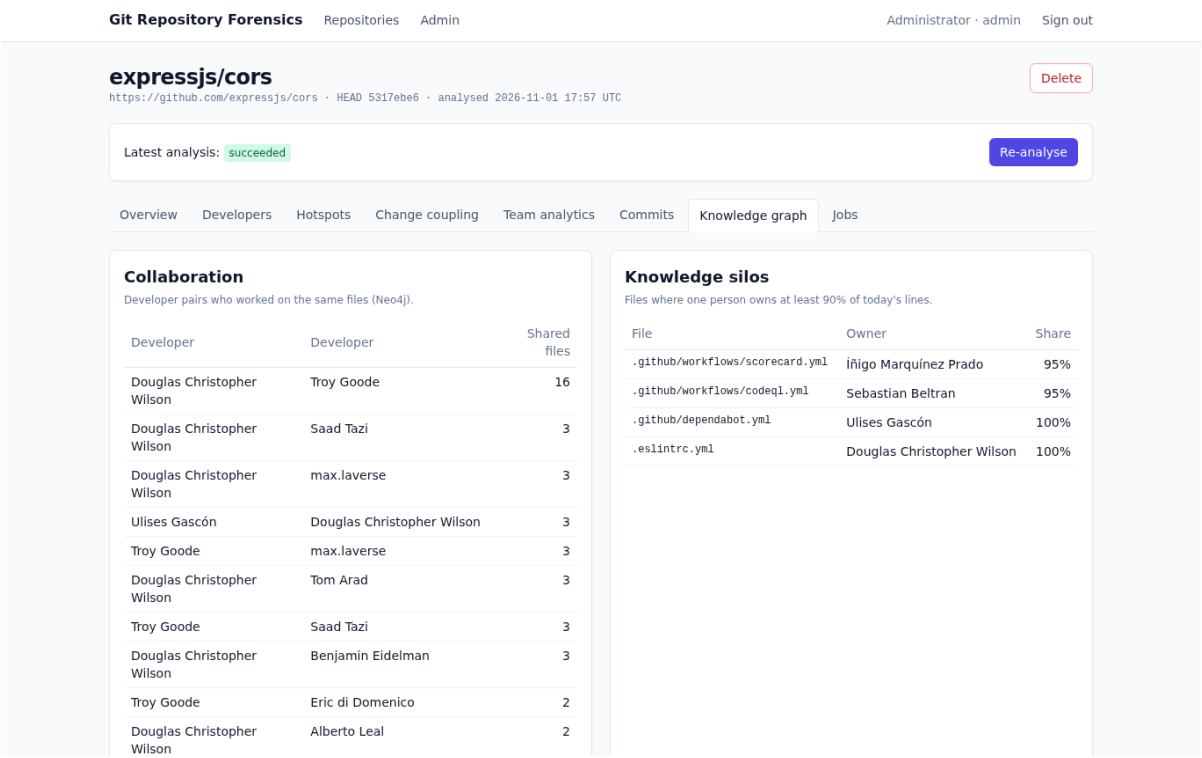


Figure 16: Knowledge graph (Neo4j) – collaboration pairs and knowledge silos

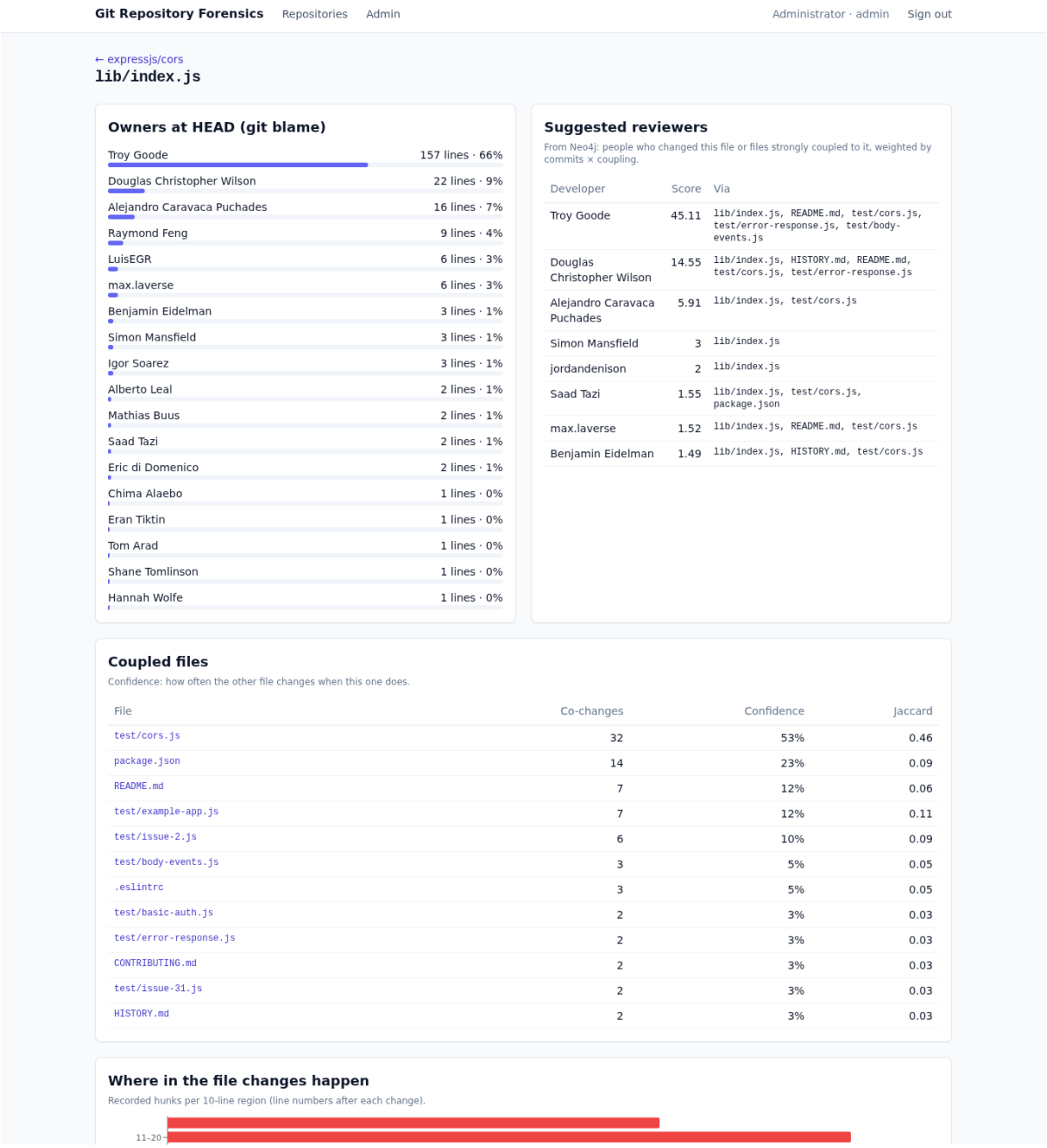


Figure 17: File detail – history, owners, coupled files and suggested reviewers

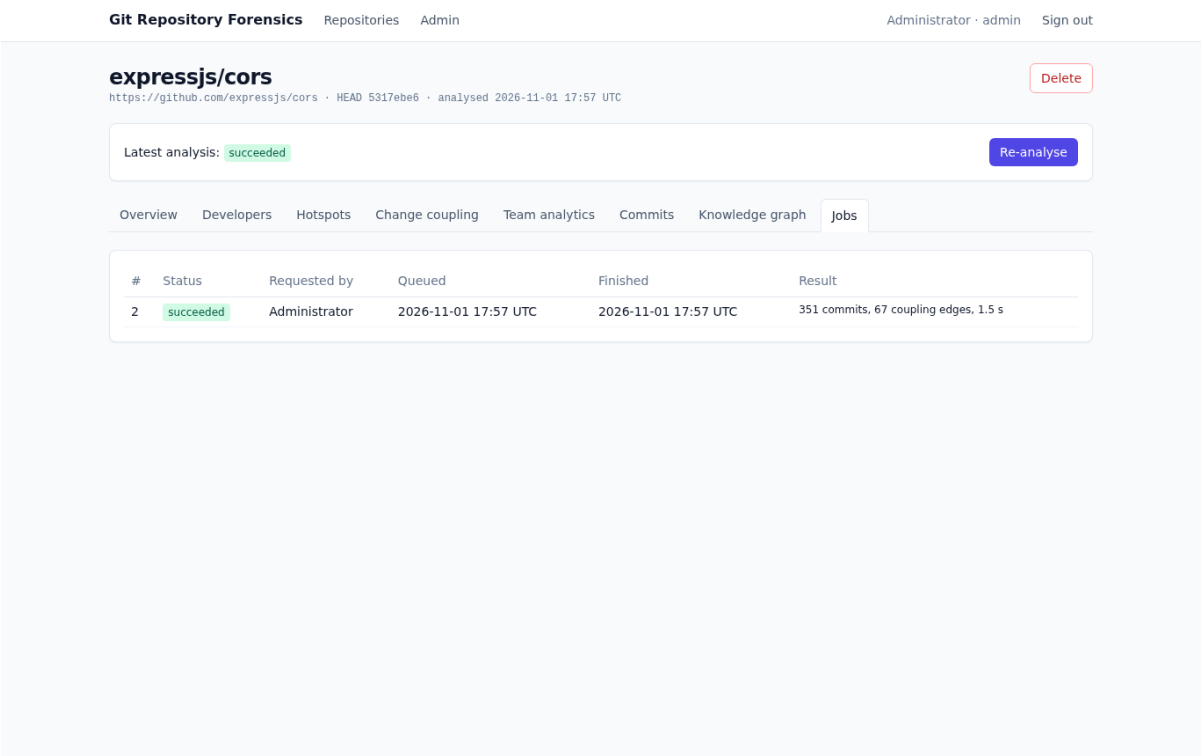


Figure 18: Analysis jobs with live progress

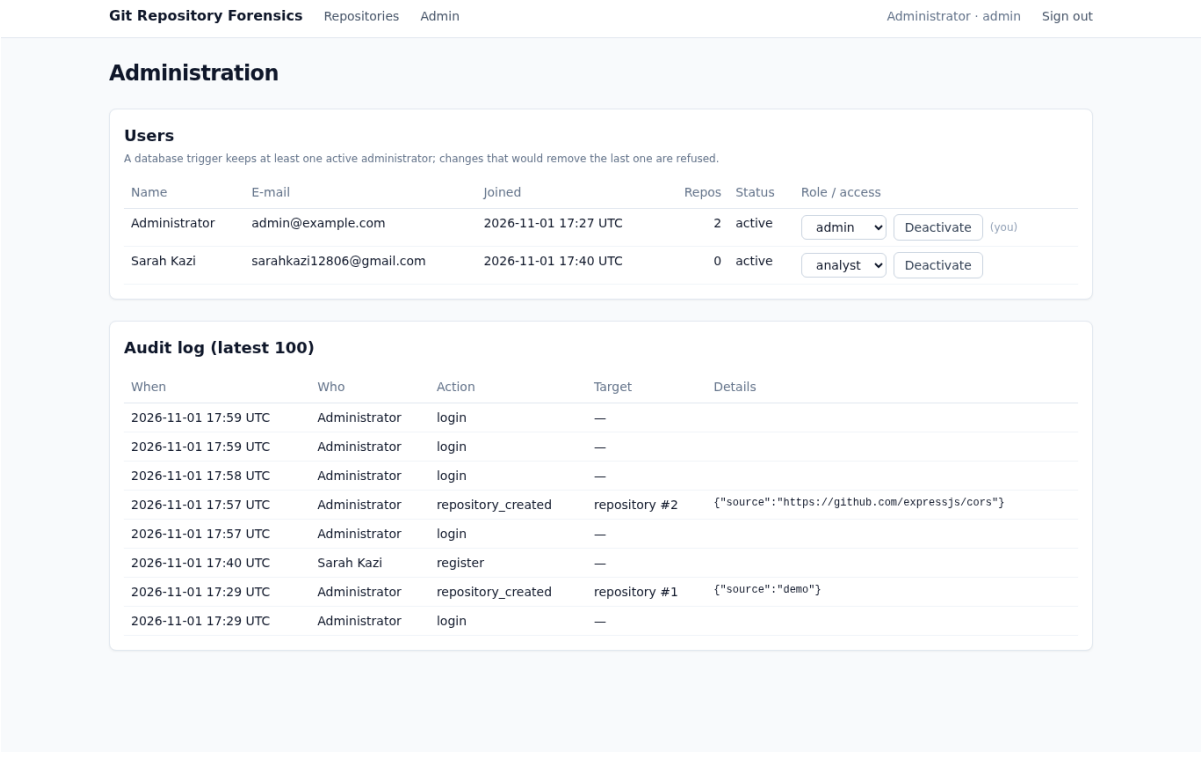


Figure 19: Administration – users, roles and audit log

9. EVALUATION RUBRICS – COMPLETION STATUS

Sl.	Rubric item	Evidence in the project	Status
1	Problem definition and requirements	Problem statement, objectives, scope and functional requirements documented (README, docs/PROJECT-REPORT.md)	Completed
2	ER model and relational schema	ER diagram, relational schema and data dictionary for 13 tables (docs/PROJECT-REPORT.md, docs/DATABASE-DESIGN.md)	Completed
3	DDL statements	Enums, tables, PK/FK, CHECK, UNIQUE, generated columns, indexes; migrations 001 and 002	Completed
4	DML statements	INSERT (incl. ON CONFLICT and bulk unnest), UPDATE, DELETE with cascades	Completed
5	Queries – joins, aggregation, nested and correlated	Q1–Q6: JOINs, GROUP BY / HAVING, CTEs, subqueries, correlated subquery, window functions	Completed
6	Views and materialised view	developer_activity, file_hotspots, monthly_activity	Completed
7	Stored function, procedure and triggers	bus_factor(), purge_expired_sessions(), three triggers	Completed
8	Transactions and concurrency control	Atomic re-analysis, SKIP LOCKED job queue, partial unique index	Completed
9	Indexing and performance	B-tree, composite, partial, expression, GIN and HNSW indexes; Redis caching	Completed
10	NoSQL / vector database	Neo4j graph (Cypher) and pgvector similarity search; Redis key-value store	Completed
11	Front end (Next.js)	Login, repositories, dashboard, file detail, jobs and admin pages with charts	Completed
12	Security and data integrity	bcrypt, hashed session tokens, rate limiting, input validation, audit log, security checklist ticked	Completed
13	Testing	Vitest: 4 test files, 58 tests (unit and integration) – recorded as passed in docs/PROJECT-REPORT.md	Completed
14	Documentation	README, architecture, API, analytics, database design, security and project report	Completed

10. GITHUB LINK

https://github.com/krista9669/git-repository-forensics